

**Relazione progetto per il corso:
Introduzione alla programmazione di smart contract
per Ethereum**

Marketplace per NFT nell'ambito di una filiera farmaceutica

Ariana Baci

Anno Accademico 2025/2026

Corso di Laurea in Informatica
Università degli Studi di Verona

10 settembre 2026

1 Interpretazione della traccia

Il progetto nasce dall'idea di sfruttare la tecnologia blockchain per garantire la tracciabilità e l'autenticità dei prodotti. In particolare, in questo caso ho pensato al caso della filiera farmaceutica, dove può essere estremamente importante avere la garanzia che un determinato prodotto derivi da uno specifico produttore. La situazione che ho rappresentato è una semplificazione di un'ipotetica situazione reale. Ci sono due contratti: uno appartiene al laboratorio che produce i farmaci, il secondo si occupa di caricare gli annunci relativi. Idealmente quindi, il laboratorio, quando un farmaco viene reso disponibile, ne crea l'NFT corrispondente e ne carica l'annuncio sul marketplace scelto per mettere in vendita i propri prodotti. Il marketplace deve sia essere autorizzato dal contratto che crea gli NFT, sia autorizzare lui stesso quel contratto a pubblicare annunci. In questo caso ho reso possibile per semplicità l'autorizzazione di un solo contratto, ma si potrebbe estendere per fare in modo che il marketplace gestisca una lista di vari produttori di farmaci.

Questo sistema risulta molto più sicuro rispetto alla semplice registrazione dei dati in un database centralizzato: grazie all'unicità degli NFT non c'è il rischio che vengano ad esempio riciclati codici identificativi per vendere prodotti contraffatti, mentre la decentralizzazione garantisce che il produttore non modifichi i dati immutabili a posteriori e che i dati non smettano di essere fruibili se cade un server.

2 Architettura del codice

Il progetto si compone di due contratti: `DrugNFT.sol` crea gli NFT che rappresentano i farmaci e `DrugMarketplace.sol` si occupa di gestirne gli annunci di vendita.

2.1 DrugNFT

`DrugNFT` è il contratto che permette di mintare i farmaci. Implementa lo standard ERC-721 (esteso tramite `ERC721URIStorage` di OpenZeppelin) per la gestione dei metadati e `Ownable` per il controllo degli accessi basato su un unico proprietario (in questo caso direttamente collegato al laboratorio produttore). Grazie allo struct apposito e al mapping associa univocamente a ogni tokenId nome, numero di lotto, data di produzione, data di scadenza, produttore, acquirente e gli indicatori di stato (`forSale`, `sold`). L'operazione di minting è riservata al proprietario del contratto (`onlyOwner`). Al suo interno,

la funzione `mintDrugNFT` esegue controlli sulle date, aumenta il valore del `tokenID` prima di chiamare *safeMint* per assicurare che non vengano duplicati, emette l'evento `DrugM`

2.2 DrugMarketplace

`DrugMarketplace` è il contratto che gestisce la compravendita dei farmaci. Utilizza `ReentrancyGuard` per proteggere da attacchi di reentrancy. Le funzioni `setDrugNFT` e `unSetDrugNFT` permettono all'amministratore (`onlyOwner`) di associare o disassociare il contratto `DrugNFT` di riferimento (che in questo caso è unico ma realisticamente un marketplace potrebbe gestirne di più). `ListDrug` consente solo al legittimo possessore dell'`NFT` di mettere in vendita il farmaco impostando un prezzo (`price > 0`). Verifica che il marketplace sia stato accreditato da `DrugNFT`, aggiorna lo stato interno (`isActive = true`), sincronizza il flag su `DrugNFT` (`markAsForSale`) ed emette l'evento `DrugListed`. `BuyDrug` permette a qualsiasi indirizzo (diverso dal venditore) di acquistare un annuncio attivo e non scaduto; trasferisce la proprietà dell'`NFT` dal venditore all'acquirente via `safeTransferFrom`, aggiorna i flag di vendita (`markAsSold`, `markAsForSale = false`) e disattiva l'annuncio, invia il pagamento al venditore tramite `call` e rimborsa automaticamente all'acquirente l'eventuale `ETH` inviato in eccesso (`msg.value > price`). `CancelListing` e `updatePrice` permettono solo al venditore di annullare un annuncio attivo (es. per imminente scadenza o ritiro dal mercato) o di aggiornarne il prezzo di vendita. `GetListing`, `getPrice`, `getSeller` permettono consultare rapidamente i dettagli degli annunci.

3 Test

I test sono specifici per i due contratti. Quelli in `typescript` sono nella cartella `"test"` mentre quelli in `solidity` si trovano in `"contracts"`.

3.1 Test in Typescript

3.1.1 DrugNFT

Inizializzazione del Contratto: Verifica che il nome (`"DrugTraceability"`) e il simbolo (`"DRUG"`) siano impostati correttamente. Conferma che il ruolo di owner appartenga al wallet del producer. Creazione/Minting (`mintDrugNFT`): Caso di successo: Verifica che il producer riesca a coniare un `NFT` e che la struct `Drug` salvata contenga i valori corretti. Verifica che l'account `other` non autorizzato fallisca se tenta di mintare un `NFT`. Verifica che la

transazione fallisca se `expirationDate` `productionDate` oppure se il farmaco ha una data di scadenza nel passato.

Interazione con il Marketplace: verifica che solo l'owner del contratto possa invocare `setMarketplace`. e che solo il marketplace autorizzato possa invocare `markAsSold` e `markAsForSale`.

3.1.2 DrugMarketplace

Sfrutta `networkHelpers` creando una fixture (`deployMarketplaceFixture`) così da partire sempre dallo stesso stato senza dover rieseguire ogni volta i vari deploy dei contratti. Verifiche svolte:

- `setDrugNFT` / `unSetDrugNFT`: verifica che solo l'amministratore possa impostare o azzerare (`address(0)`) l'indirizzo del contratto `DrugNFT`.
- `listDrug`: verifica che la pubblicazione di un annuncio sia permessa solo al proprietario dell'`NFT`.
- `buyDrug`. Si verifica l'impossibilità di: acquistare annunci revocati/inattivi, acquistare un farmaco di cui si è proprietari, acquistare con fondi insufficienti o farmaci scaduti. In caso di successo invece, si verifica che un acquisto valido trasferisca il token, aggiorni lo stato a `isSold = true` ed esegua la transazione.
- `updatePrice`, `getListing`, etc: controlla che solo il venditore possa modificare il prezzo con `updatePrice` e la corretta estrazione dei dati da parte delle funzioni `getPrice`, `getSeller` e `getListing`.
-

3.2 Test in Solidity

3.2.1 DrugNFT

- Configurazione (`setUp`): Istanza `DrugNFT` impostando l'indirizzo fittizio `producer (0x1111)` come proprietario iniziale.
- Minting : simula la chiamata da parte del produttore autorizzato sfruttando `vm.prank(producer)`, crea un `NFT`, verifica che il `tokenId` generato sia 0 (primo token mintato) e che le variabili interne alla struct `Drug` (`name`, `lotNumber`, `producer`, `sold = false`) corrispondano esattamente ai dati inseriti.
- Scadenze (`testIsExpired`) : sfruttando `vm.warp` simulando l'avanzare del tempo dell'`EVM`, face

3.2.2 DrugMarketplace

- Configurazione (setUp): Esegue il deploy di entrambi i contratti (DrugNFT e DrugMarketplace). Stabilisce i riferimenti incrociati (setMarketplace su DrugNFT e setDrugNFT su DrugMarketplace) usando `vm.prank(producer)`.
- Test flusso di vendita (testVendita): usando `vm.startPrank(producer)`, il produttore conia il token 0 e crea l'annuncio su DrugMarketplace per 100 wei. Con `vm.deal(buyer, 1 ether)` aggiunge i fondi necessari all'acquirente, esegue poi l'acquisto con `marketplace.buyDrug` sotto `vm.startPrank(buyer)`.
- Accerta il trasferimento di proprietà dell'NFT in favore dell'acquirente (`drugNFT.ownerOf(tokenId) == buyer`), estrae la tupla da `marketplace.getListing` tramite la sintassi a parentesi tonde `()` e verifica che `isActive` e `isForSale` siano resettati a `false`.